

Universidad Andina del Cusco

Escuela Profesional de Ingeniería de Sistemas

Ingeniería de Software — SIS021 | IX Ciclo | Plan 2025 | 2026-I

ACTIVIDAD 2.2 — UNIDAD II

Modelos UML del Sistema

Sistema de Atención a Estudiantes — SASE

Equipo	Equipo Delta
Código	SIS021
Semestre	2026-I
Fecha de entrega	16 de octubre de 2026
Docente	Dr. Lornel Antonio Rivas Mago
Integrantes	AM — Ana Mamani Quispe (Product Owner) CR — Carlos Ríos Huanca (Scrum Master) LT — Lucía Ttito Ccari (Desarrollador) JP — Jorge Palomino Vargas (Desarrollador)
Nombre del archivo	SIS021_Act2-2_EquipoDelta_2026I.pdf

Actividad previa: Act 2.1 — Especificación de Requisitos del SASE

1. Introducción

Este documento presenta el conjunto de modelos UML elaborados para el Sistema de Atención a Estudiantes (SASE) de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Andina del Cusco. El SASE centraliza el registro, priorización y canalización de solicitudes de apoyo académico, administrativo y de otra índole, permitiendo al estudiante hacer seguimiento en tiempo real del estado de su caso.

Los modelos parten del documento de especificación de requisitos elaborado en la Actividad 2.1 y del backlog de producto definido en la Actividad 1.3. Se presentan cuatro vistas complementarias del sistema, siguiendo la clasificación de Sommerville (2016, cap. 5): contexto, casos de uso, estructura y comportamiento. Cada modelo está construido con notación UML 2.5.1 (OMG, 2017).

Modelo	Tipo de vista	Pregunta que responde	Sección
Diagrama de contexto	Externa	¿Cómo se relaciona el sistema con su entorno?	2
Diagrama de casos de uso	Interacción	¿Qué hace el sistema y para quién?	3
Diagrama de clases	Estructural	¿De qué está compuesto el sistema?	4
Diagrama de estados	Comportamiento	¿En qué estados puede estar la entidad principal?	5

2. Diagrama de Contexto del Sistema

El diagrama de contexto sitúa al SASE en su entorno operativo. Muestra los cinco actores que interactúan directamente con el sistema y el sistema externo (Correo Institucional) con el que se integra para el envío de notificaciones. Los flujos continuos representan interacciones directas; los flujos discontinuos, respuestas o notificaciones enviadas por el sistema.

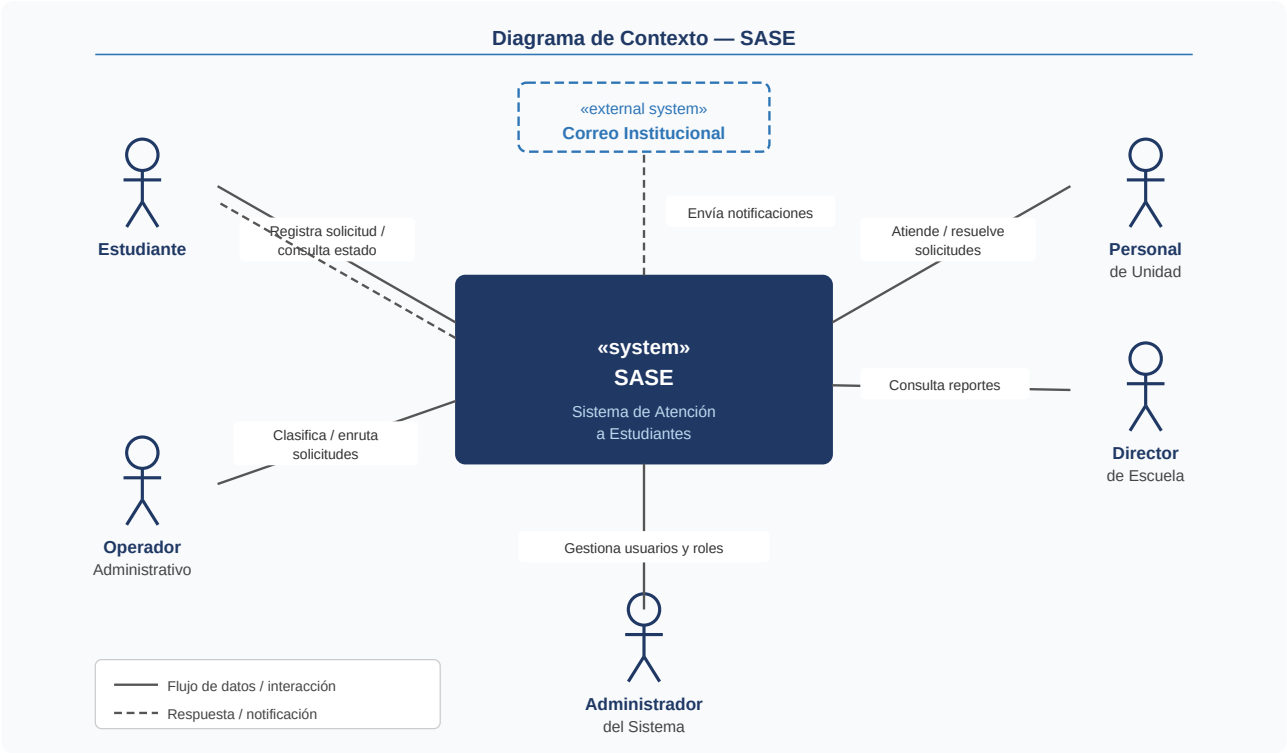


Figura 1. Diagrama de contexto del SASE. Elaborado con notación de diagrama de contexto UML. Actores: Estudiante, Operador Administrativo, Personal de Unidad, Director de Escuela, Administrador del Sistema. Sistema externo: Correo Institucional.

Descripción de actores y flujos principales

Actor / Sistema	Tipo	Flujo principal hacia / desde el SASE
Estudiante	Primario	Registra solicitudes, consulta estado de sus casos.
Operador Administrativo	Primario	Clasifica solicitudes entrantes y las deriva a la unidad responsable.
Personal de Unidad	Primario	Visualiza solicitudes asignadas a su área, registra resolución o redirige.
Director de Escuela	Secundario	Consulta reportes agregados por tipo, período y estado.
Administrador del Sistema	Secundario	Gestiona cuentas de usuario y asignación de roles de acceso.
Correo Institucional	Sistema externo	Recibe notificaciones generadas por el SASE ante cambios de estado.

3. Diagrama de Casos de Uso

El diagrama de casos de uso representa las diez historias de usuario del backlog (Actividad 1.3) como casos de uso UML, agrupadas en las cuatro épicas del sistema. Se incluyen dos relaciones «include»: HU01 incluye HU05 porque el registro de solicitud desencadena la asignación automática de prioridad; y HU07 incluye HU03 porque registrar la resolución siempre dispara la notificación al estudiante.

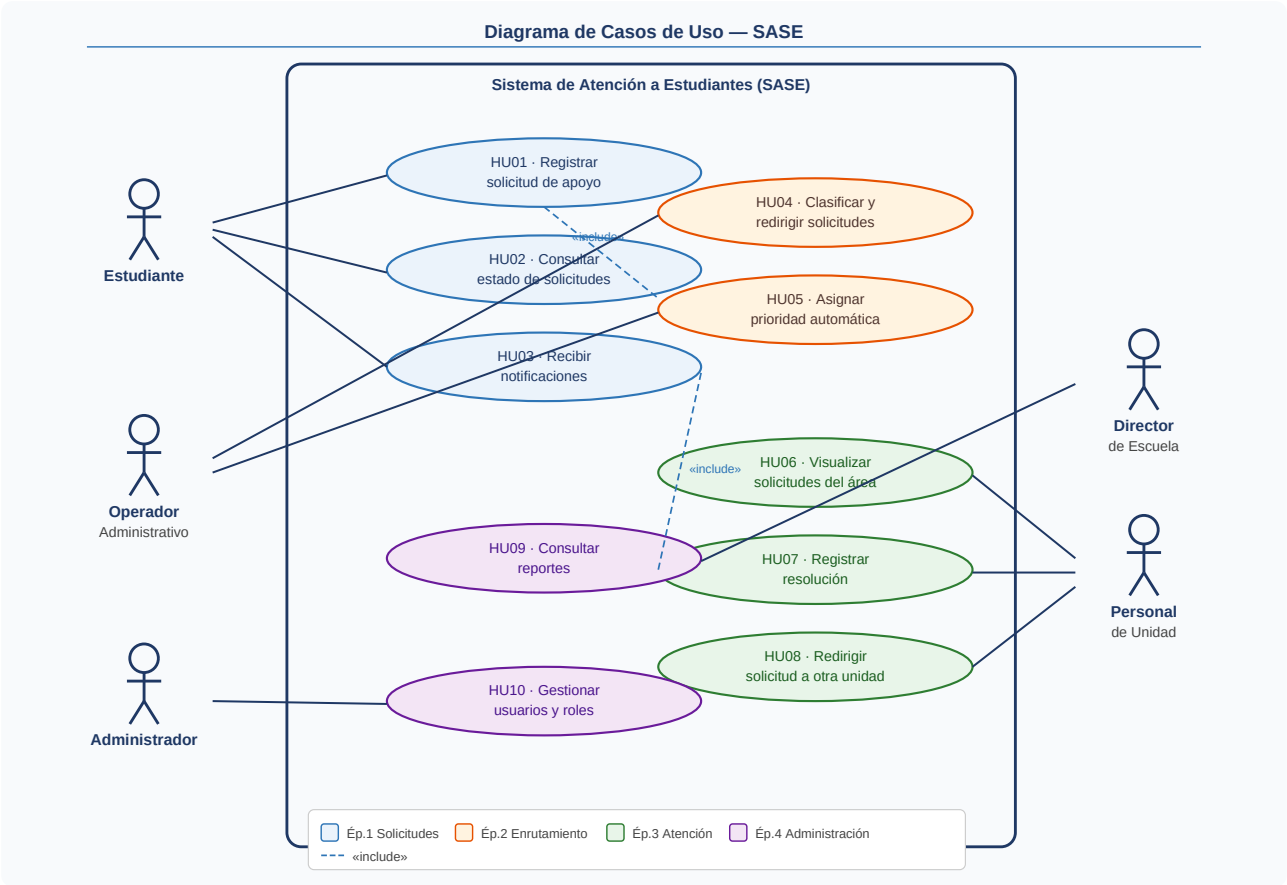


Figura 2. Diagrama de casos de uso del SASE. 10 casos de uso trazables al backlog (Actividad 1.3). Relaciones «include» marcadas con línea discontinua.

Descripción breve de casos de uso

ID	Nombre del caso de uso	Actor principal	Objetivo	Precondición
HU01	Registrar solicitud de apoyo	Estudiante	Documentar formalmente una necesidad del estudiante.	Usuario autenticado.
HU02	Consultar estado de solicitudes	Estudiante	Ver el historial y estado actual de sus casos.	Solicitud registrada.
HU03	Recibir notificaciones	Estudiante	Ser informado de cambios sin consultar manualmente.	Correo institucional activo.
HU04	Clasificar y redirigir solicitudes	Operador	Derivar cada solicitud a la unidad responsable.	Solicitud en estado Pendiente.

HU05	Asignar prioridad automática	Sistema	Aplicar reglas de prioridad al registrar solicitud.	«include» desde HU01.
HU06	Visualizar solicitudes del área	Personal Unidad	Gestionar solicitudes asignadas dentro de plazos.	Solicitud derivada a la unidad.
HU07	Registrar resolución	Personal Unidad	Cerrar formalmente un caso con respuesta.	Solicitud en estado En Proceso.
HU08	Redirigir a otra unidad	Personal Unidad	Evitar demoras cuando la solicitud no corresponde al área.	Solicitud En Proceso.
HU09	Consultar reportes	Director	Tomar decisiones con datos agregados del sistema.	Acceso a módulo de reportes.
HU10	Gestionar usuarios y roles	Administrador	Controlar acceso según responsabilidades.	Acceso de administrador.

4. Diagrama de Clases

El diagrama de clases representa la estructura estática del SASE. Se identificaron seis clases a partir de las entidades mencionadas en los requisitos: **Usuario** (superclase), **Estudiante**, **Operador** y **PersonalUnidad** (subclases), **Solicitud** (entidad central del dominio), **DetalleSolicitud** y **Notificacion**. El diagrama incluye las tres relaciones UML requeridas.

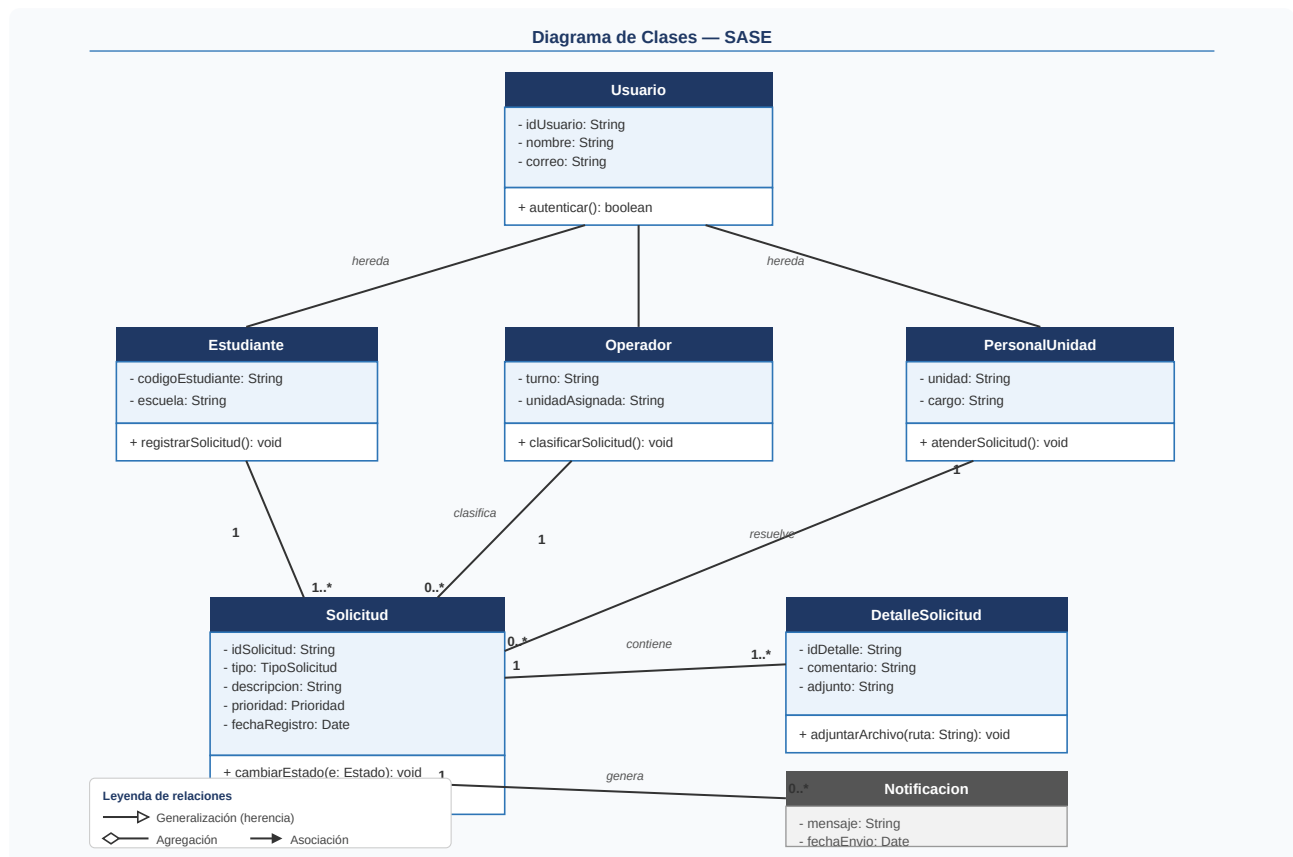


Figura 3. Diagrama de clases del SASE. Notación UML 2.5.1. Incluye generalización, asociación con multiplicidad y agregación.

Justificación de las relaciones de diseño

Relación	Tipo UML	Justificación de diseño
Usuario → Estudiante / Operador / PersonalUnidad	Generalización (herencia)	Los tres actores comparten atributos comunes (idUsuario, nombre, correo) y el método autenticar(). La herencia evita duplicación y permite tratar a cualquier usuario polimórficamente en el módulo de autenticación.
Estudiante (1) → Solicitud (1..*)	Asociación con multiplicidad	Un estudiante puede registrar una o más solicitudes a lo largo del tiempo (mínimo 1 para que la asociación sea útil). Una solicitud pertenece a exactamente un estudiante: no existe solicitud anónima en el sistema.

Operador (1) → Solicitud (0..*)	Asociación con multiplicidad	Un operador puede clasificar cero o muchas solicitudes en su turno. La multiplicidad 0..* refleja que puede existir un operador sin solicitudes pendientes asignadas en un momento dado.
PersonalUnidad (1) → Solicitud (0..*)	Asociación con multiplicidad	El personal de una unidad resuelve las solicitudes derivadas a su área. La multiplicidad 0..* permite que en un período no haya solicitudes asignadas.
Solicitud (1) ■■■ DetalleSolicitud (1..*)	Agregación	Una solicitud se compone de uno o más detalles (descripción inicial, adjuntos, comentarios adicionales). Se usa agregación (no composición) porque los detalles podrían consultarse independientemente para fines de auditoría, aunque en la práctica están fuertemente ligados a la solicitud.
Solicitud (1) → Notificacion (0..*)	Asociación con multiplicidad	Cada cambio de estado de una solicitud puede generar una notificación. La multiplicidad 0..* refleja que no todos los estados disparan notificación (por ejemplo, la asignación de prioridad interna no notifica al estudiante).

5. Diagrama de Estados — Solicitud

La entidad principal del dominio es **Solicitud**: es el objeto con el ciclo de vida más complejo, el que transita por más estados y el que concentra las interacciones de todos los actores. El diagrama modela seis estados posibles y las transiciones disparadas por eventos del sistema o acciones de los actores.

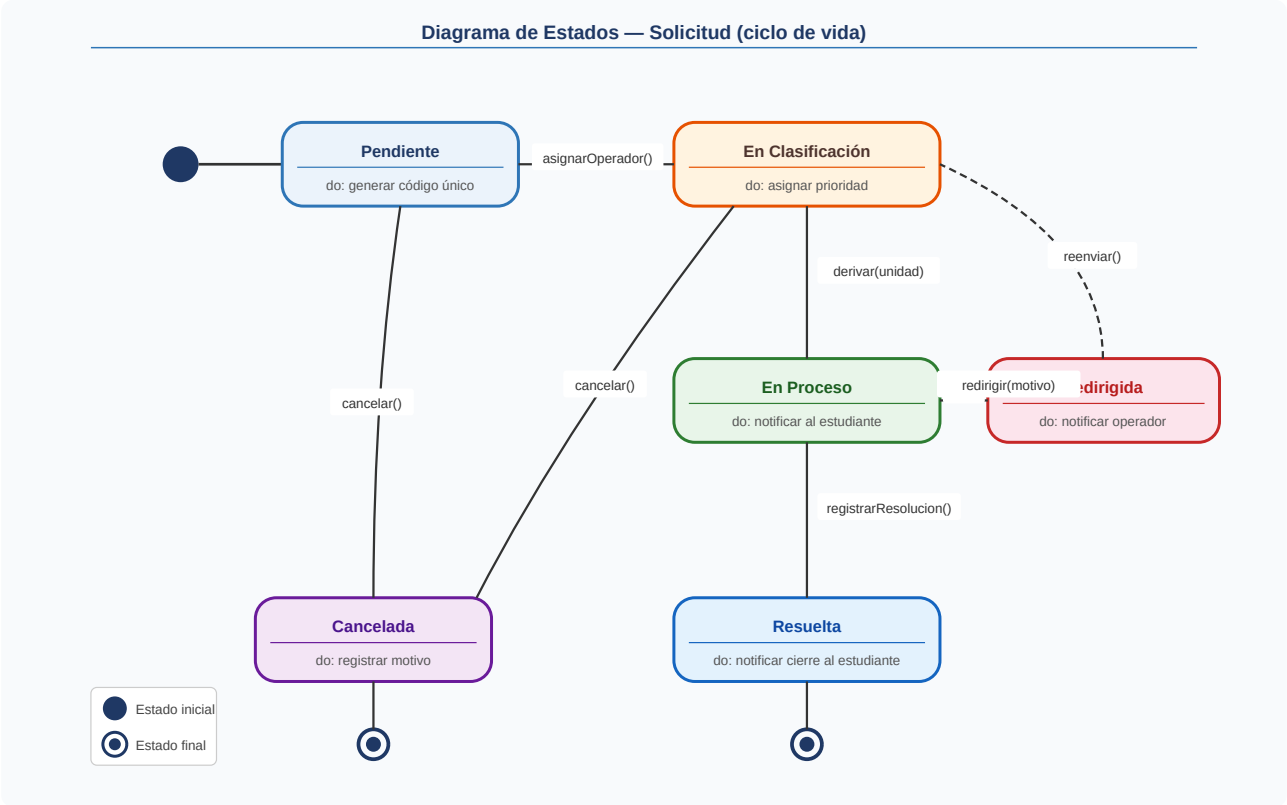


Figura 4. Diagrama de estados de la clase *Solicitud*. Notación UML 2.5.1 (statechart). Estado inicial (●), seis estados intermedios y dos estados finales posibles (⦿).

Descripción de estados y transiciones

Estado	Significado en el dominio	Acción interna (do:)	Transiciones de salida
Pendiente	La solicitud fue registrada por el estudiante y aguarda clasificación.	Genera código único de seguimiento.	asignarOperador() → En Clasificación cancelar() → Cancelada
En Clasificación	El operador revisa la solicitud, asigna prioridad y selecciona la unidad.	Asigna nivel de prioridad automático.	derivar(unidad) → En Proceso cancelar() → Cancelada
En Proceso	La unidad receptora tiene la solicitud asignada y trabaja en su resolución.	Notifica al estudiante el cambio de estado.	registrarResolucion() → Resuelta redirigir(motivo) → Redirigida
Redirigida	La unidad rechazó la asignación por no corresponder a su área.	Notifica al operador para reasignación.	reenviar() → En Clasificación

Resuelta	La unidad registró la respuesta y cerró formalmente el caso.	Notifica el cierre al estudiante.	→ Estado final
Cancelada	La solicitud fue anulada (por el estudiante o por política del sistema).	Registra motivo de cancelación.	→ Estado final

6. Análisis de Consistencia entre Modelos

Los cuatro modelos son consistentes entre sí y con los requisitos de la Actividad 2.1 en los siguientes puntos:

Actores ↔ Casos de uso	Los cinco actores del diagrama de contexto (Estudiante, Operador, Personal de Unidad, Director, Administrador) aparecen exactamente como actores en el diagrama de casos de uso. No hay actor en el contexto sin al menos un caso de uso asociado.
Casos de uso ↔ Backlog	Los diez casos de uso son trazables uno a uno a las diez historias de usuario del backlog (HU01–HU10) de la Actividad 1.3. La prioridad Highest del backlog corresponde a los casos de uso de las épicas 1 y 2, que modelan el flujo principal.
Casos de uso ↔ Clases	Cada caso de uso de la épica 1 involucra a la clase Estudiante y la clase Solicitud. La épica 2 involucra a Operador y Solicitud. La épica 3 a PersonalUnidad y Solicitud. La épica 4 a Director/Administrador y el sistema en general.
Diagrama de clases ↔ Diagrama de estados	La entidad Solicitud modelada con sus atributos (estado, prioridad, tipo) en el diagrama de clases corresponde exactamente a la clase cuyo ciclo de vida se modela en el diagrama de estados. Los métodos cambiarEstado() y asignarUnidad() de la clase Solicitud corresponden a las transiciones del diagrama de estados.
Diagrama de estados ↔ HU03 (notificaciones)	El diagrama de estados muestra que tres transiciones generan notificaciones: En Clasificación → En Proceso, En Proceso → Resuelta, y Redirigida. Esto es consistente con HU03, que especifica que el sistema notifica al correo institucional ante cualquier cambio de estado relevante.

7. Referencias bibliográficas

Equipo Delta. SIS021_Act1-3_EquipoDelta_2026I.pdf — Backlog de Producto: Sistema de Atención a Estudiantes (SASE). Actividad 1.3, Unidad I. Ingeniería de Software — SIS021, Universidad Andina del Cusco, 2026-I.

Equipo Delta. SIS021_Act2-1_EquipoDelta_2026I.pdf — Documento de Especificación de Requisitos: SASE. Actividad 2.1, Unidad II. Ingeniería de Software — SIS021, Universidad Andina del Cusco, 2026-I.

IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) v4.0.* IEEE Computer Society.
<https://www.computer.org/education/bodies-of-knowledge/software-engineering>

Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1.*
<https://www.omg.org/spec/UML/>

Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson. Capítulos 5 (§5.1–5.5) y 7 (§7.1).